

Odd Even Linked List

Structured DSA Study Notebook — Revised Code Presentation

Revision made: The code sections have been reformatted so every Java statement appears line-by-line in a proper code block. Comments are retained and placed directly above the statement they explain. No code is presented as paragraph text.

1. Introduction

The problem rearranges a linked list by node position. Nodes at odd positions are placed first, followed by nodes at even positions. The relative order inside each group is preserved.

2. Problem Statement

Given the head of a singly linked list, group all nodes at odd indices together followed by all nodes at even indices. The first node is at index 1. Return the rearranged list.

```
Example:
Original:
1 -> 2 -> 3 -> 4 -> 5 -> NULL

Result:
1 -> 3 -> 5 -> 2 -> 4 -> NULL
```

3. Required Concepts

3.1 Position vs. Value

```
// Odd/even refers to the POSITION of a node.

// Example:
Position: 1  2  3  4  5
Value:    10 20 30 40 50

Odd positions: 10, 30, 50
Even positions: 20, 40
```

3.2 Pointer Traversal

```
// Move to the next node:
current = current.next;

// Skip one node and move two positions:
current = current.next.next;
```

4. Approaches

Two approaches are covered: a brute-force approach using an external data structure, and the optimized approach that rearranges links directly.

5. Approach 1 — Brute Force

5.1 Idea

Traverse odd positions first and store their values. Then traverse even positions and store their values. Finally, write the collected values back into the linked list.

5.2 Pseudocode

```
START

Create an empty array
```

```

Set temp = head

WHILE temp is not NULL
    Add temp.data to array
    Move temp by two nodes
    IF temp is NULL
        STOP
    END IF
END WHILE

Set temp = head.next

WHILE temp is not NULL
    Add temp.data to array
    Move temp by two nodes
END WHILE

Set temp = head

FOR every value in array
    temp.data = value
    Move temp to next node
END FOR

Return head

END

```

5.3 Original Core Algorithm

```

// Traverse odd positions
temp = head;
while(temp != null){
    array.add(temp.data);
    if(temp.next != null){
        temp = temp.next.next;
    }else{
        temp = null;
    }
}

// Traverse even positions
temp = head.next;
while(temp != null){
    array.add(temp.data);
    if(temp.next != null){
        temp = temp.next.next;
    }else{
        temp = null;
    }
}

```

5.4 Complete Java Code

```

class Solution {

    public ListNode oddEvenList(ListNode head) {

        // Handle empty list
        if(head == null){
            return null;
        }

        // Store node values
        List<Integer> array = new ArrayList<>();

        // Start from the head
        ListNode temp = head;

        // Traverse odd-position nodes
        while(temp != null){

```

```

        // Store current node value
        array.add(temp.data);

        // Move two positions ahead
        if(temp.next != null){
            temp = temp.next.next;
        }
        else{
            temp = null;
        }
    }

    // Start from the second node
    temp = head.next;

    // Traverse even-position nodes
    while(temp != null){

        // Store current node value
        array.add(temp.data);

        // Move two positions ahead
        if(temp.next != null){
            temp = temp.next.next;
        }
        else{
            temp = null;
        }
    }

    // Start again from the head
    temp = head;

    // Copy values back into the list
    for(int value : array){

        // Replace current node value
        temp.data = value;

        // Move to next node
        temp = temp.next;
    }

    // Return original head
    return head;
}
}

```

5.5 Static Visual Representation

Original:

NULL <- not applicable for singly list

```

HEAD
|
v
[1] -> [2] -> [3] -> [4] -> [5] -> NULL

```

Read odd positions:

```
[1] -> [3] -> [5]
```

Read even positions:

```
[2] -> [4]
```

Collected order:

[1, 3, 5, 2, 4]

Write values back:

HEAD

|

v

[1] -> [3] -> [5] -> [2] -> [4] -> NULL

5.6 Complexity

Time Complexity:

$O(n)$

Extra Space:

$O(n)$

Reason:

The array stores all n node values.

6. Approach 2 — Optimized Two-Pointer Approach

6.1 Idea

Instead of storing values in an external array, maintain two linked sections: an odd-position chain and an even-position chain. At the end, connect the odd chain to the saved beginning of the even chain.

6.2 Pointer Setup

```
// Odd pointer starts at the first node
odd = head;

// Even pointer starts at the second node
even = head.next;

// Save the beginning of the even list
evenHead = head.next;
```

6.3 Pseudocode

```
START

If head is NULL OR head.next is NULL
    Return head

Set odd = head
Set even = head.next
Set evenHead = head.next

WHILE even is not NULL AND even.next is not NULL

    odd.next = odd.next.next

    even.next = even.next.next

    odd = odd.next

    even = even.next

END WHILE

odd.next = evenHead

Return head

END
```

6.4 Original Core Code From the Transcript

The transcript presents the optimized algorithm through these core link operations. They are kept separately from the complete Java solution.

```
// Connect current odd node to next odd node
odd.next = odd.next.next;

// Connect current even node to next even node
even.next = even.next.next;

// Move odd pointer
odd = odd.next;

// Move even pointer
even = even.next;

// Attach even list after odd list
odd.next = evenHead;
```

6.5 Complete Java Code — Line by Line

```
class Solution {
```

```

public ListNode oddEvenList(ListNode head) {

    // Handle empty list or single-node list
    if(head == null || head.next == null){
        return head;
    }

    // Pointer for odd-position nodes
    ListNode odd = head;

    // Pointer for even-position nodes
    ListNode even = head.next;

    // Save the first even-position node
    ListNode evenHead = head.next;

    // Continue while an even node and its next node exist
    while(even != null && even.next != null){

        // Connect current odd node to the next odd node
        odd.next = odd.next.next;

        // Connect current even node to the next even node
        even.next = even.next.next;

        // Move odd pointer to the next odd node
        odd = odd.next;

        // Move even pointer to the next even node
        even = even.next;
    }

    // Connect the end of odd list to the start of even list
    odd.next = evenHead;

    // Return the original head
    return head;
}

```

6.6 Static Visual Representation

Before:

```

HEAD
|
v
[1] -> [2] -> [3] -> [4] -> [5] -> NULL
|       |
odd     even

```

After separating links:

```

ODD LIST
[1] -> [3] -> [5] -> NULL

EVEN LIST
[2] -> [4] -> NULL

```

Final connection:

```

HEAD
|
v
[1] -> [3] -> [5] -> [2] -> [4] -> NULL

evenHead
|
v
[2] -> [4] -> NULL

```

6.7 Why the Pointer Moves Are Written This Way

```
// First change the link
odd.next = odd.next.next;

// Then move to the node reached by that new link
odd = odd.next;

// First change the link
even.next = even.next.next;

// Then move to the node reached by that new link
even = even.next;
```

6.8 Loop Condition

```
while(even != null && even.next != null){

    // odd and even links are updated here
}
```

The even pointer stays ahead of the odd pointer, so the loop condition is checked using the even side.

7. Edge Cases

Case 1: Empty list
head = null
Result = null

Case 2: One node
1 -> NULL
Result = 1 -> NULL

Case 3: Two nodes
1 -> 2 -> NULL
Result = 1 -> 2 -> NULL

Case 4: Odd number of nodes
1 -> 2 -> 3 -> 4 -> 5
Result = 1 -> 3 -> 5 -> 2 -> 4

Case 5: Even number of nodes
1 -> 2 -> 3 -> 4
Result = 1 -> 3 -> 2 -> 4

8. Complexity Comparison

Approach	Time	Extra Space	Main Idea
Brute Force	O(n)	O(n)	Store values externally
Optimized	O(n)	O(1)	Rearrange links directly

9. Quick Revision

Brute Force:

1. Collect odd-position values.
2. Collect even-position values.
3. Write values back.
4. O(n) extra space.

Optimized:

1. odd = head.
2. even = head.next.
3. Save evenHead.
4. Rewire odd links.
5. Rewire even links.
6. Move odd and even.

7. Connect odd tail to evenHead.
8. Return head.
9. $O(1)$ extra space.

10. Conclusion

The optimized solution performs the rearrangement directly through node links. The key operations are the four pointer updates inside the loop and the final connection from the odd list to evenHead.